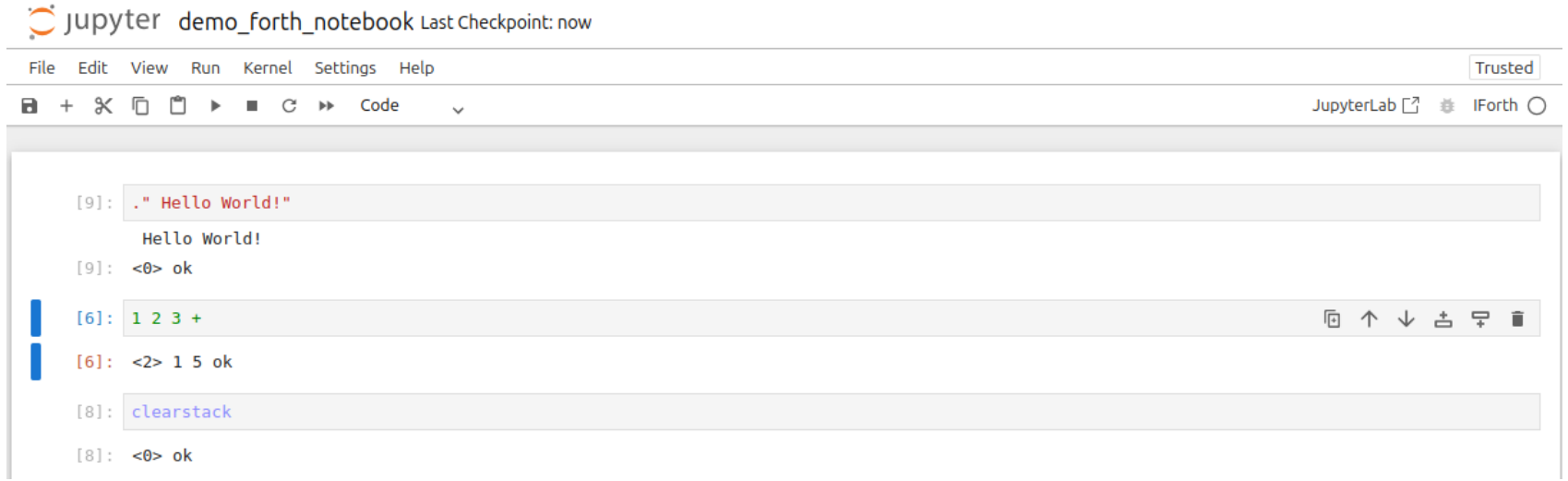
Building a Jupyter Kernel

How Python can bring another language to the notebook

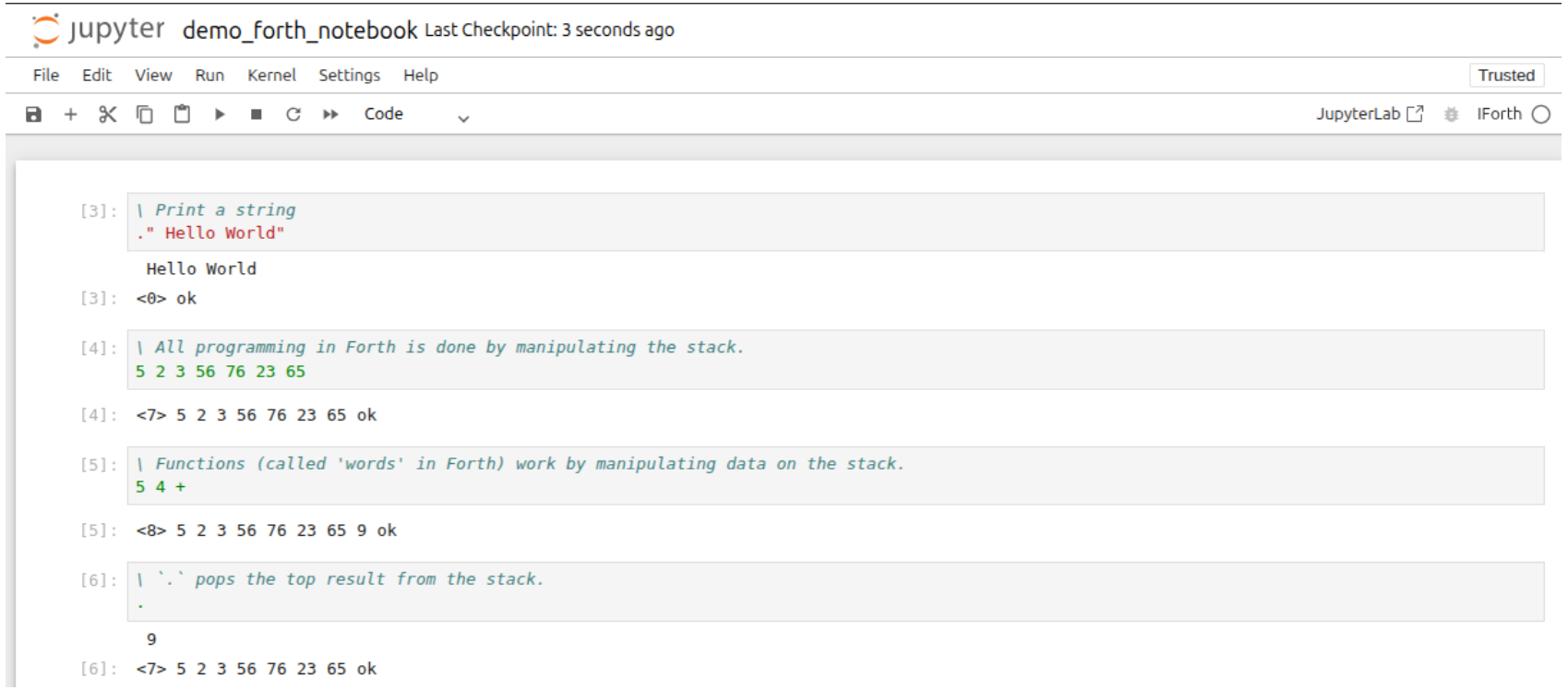
Using case study of IForth - a kernel for Forth language

1

Jupyter Notebook Examples - Python and Forth



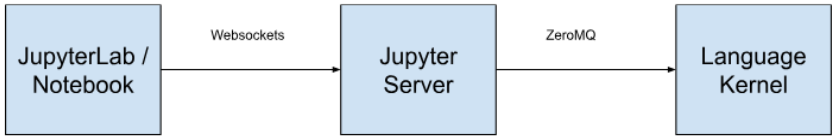
Forth in 30 seconds



See more syntax examples: https://learnxinyminutes.com/forth

3

The message flow



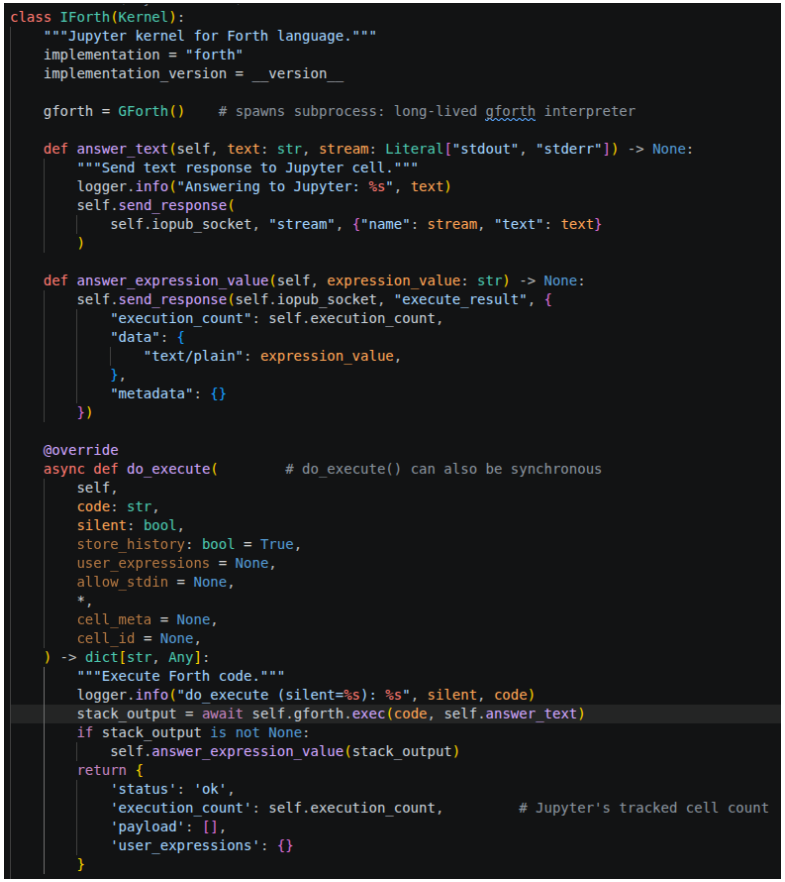
The Jupyter Server communicates with the kernel over 5 ZeroMQ channels:

- Shell: Main request/reply loop, frontend sends code to run here.
- IOPub: Broadcast channel. Kernel can publish stdout, stderr, HTML, etc.
- Stdin: Kernel can request user input.
- Control: High-priority commands (eg. Shutdown, Interrupt) that bypass execution queue.
- Heartbeat: Simple ping/pong socket, check kernel is still alive.

Source: https://www.datahaskell.org/blog/2025/11/25/a-tale-of-two-kernels.html#the-jupyter-kernel-architecture

4

Subclass Jupyter Kernel



```python
class IForth(Kernel):
    """Jupyter kernel for Forth language."""
    implementation = "forth"
    implementation_version = __version__

    gforth = GForth()    # spawns subprocess: long-lived gforth interpreter

    def answer_text(self, text: str, stream: Literal["stdout", "stderr"]) -> None:
        """Send text response to Jupyter cell."""
        logger.info("Answering to Jupyter: %s", text)
        self.send_response(
            self.iopub_socket, "stream", {"name": stream, "text": text}
        )

    def answer_expression_value(self, expression_value: str) -> None:
        self.send_response(self.iopub_socket, "execute_result", {
            "execution_count": self.execution_count,
            "data": {
                "text/plain": expression_value,
            },
            "metadata": {}
        })

    @override
    async def do_execute(        # do_execute() can also be synchronous
        self,
        code: str,
        silent: bool,
        store_history: bool = True,
        user_expressions = None,
        allow_stdin = None,
        *,
        cell_meta = None,
        cell_id = None,
    ) -> dict[str, Any]:
        """Execute Forth code."""
        logger.info("do_execute (silent=%s): %s", silent, code)
        stack_output = await self.gforth.exec(code, self.answer_text)
        if stack_output is not None:
            self.answer_expression_value(stack_output)
        return {
            'status': 'ok',
            'execution_count': self.execution_count,         # Jupyter's tracked cell count
            'payload': [],
            'user_expressions': {}
        }
```

5

Register the custom kernel with Jupyter

```
# dynamically generate kernel.json so that correct Python path is used
kernel_spec = {
    "argv": [
        # path to python (of the venv where forth_kernel is installed)
        # allows it to run even if Jupyter is installed in a different venv
        sys.executable,
        "-m",
        "forth_kernel",
        "-f",
        "{connection_file}"
    ],
    "display_name": "IForth",
    "language": "Forth",
    "codemirror_mode": "text",
    "interrupt_mode": "message",
    "name": "Forth"
}

script_dir = Path(__file__).parent.resolve()
logger.info('Script Dir: %s', script_dir)
with (script_dir / 'kernel.json').open('w') as f:
    json.dump(kernel_spec, f, indent=4)

try:
    install_kernel_spec(str(script_dir), 'forth', replace=True, user=args.user)
    logger.info('Successfully installed jupyter kernel for Forth.')
except PermissionError as e:
    logger.error(
        'Failed to install jupyter kernel for Forth: %s\n' +
        'Try installing as user instead of root: python -m forth_kernel.self_install --user',
        e
    )
except Exception as e:
    logger.error('Failed to install jupyter kernel for Forth: %s', e)
```


6

Automated Tests using module jupyter_kernel_test

```python
class MyKernelTests(jupyter_kernel_test.KernelTests):
    kernel_name = "forth"
    language_name = "forth"
    file_extension = ".4th"

    code_hello_world = '." hello, world"'
    code_stderr = 'clearstack .'      # stack underflow error
    code_execute_result = [
        {'code': '1 2 3 .', 'result': '<2> 1 2 ok'},
    ]

    def setUp(self):
        self.flush_channels()

    def test_multiline_cell_is_prompt(self):
        """Regression: each line used to cost a full read timeout."""
        start = time.monotonic()
        reply, _ = self.execute_helper(code='1 .\n2 .\n3 .\n4 .\n5 .')
        self.assertEqual(reply['content']['status'], 'ok')
        self.assertLess(time.monotonic() - start, 5)

    def test_long_output_not_truncated(self):
        """Regression: output was cut off when it exceeded the read timeout."""
        _, msgs = self.execute_helper(code=': countup 200 0 do i . loop ; countup')
        out = ''.join(m['content']['text'] for m in msgs
                      if m['msg_type'] == 'stream'
                      and m['content']['name'] == 'stdout')
        self.assertIn('199', out)

if __name__ == "__main__":
    import unittest
    unittest.main()
```


7

Check out IForth for an example Jupyter kernel implementation.

And maybe try Forth - it's an interesting language :)

Repo: https://github.com/sohang3112/iforth

Thank You!

PS: This presentation was made using https://docs.racket-lang.org/slideshow/

Questions?

8